Indian Institute of Technology Palakkad

CS5024: Geometric CV

Mini-Project 1: Estimating Camera Calibration

Submitted By:

P. Chakradhar Reddy 112201022
Vuppula Hemanth 142201020

September 2, 2025

Contents

1 Objective and Overview

The primary objective of this project is to determine a camera's 3x4 perspective projection matrix (M). This matrix is a cornerstone of 3D computer vision, defining the mathematical transformation that maps points from a 3D world coordinate system onto a 2D image plane.

Our methodology leverages a provided RGB image and its corresponding depth map from the NYU Depth V2 dataset. The core task is to solve for the projection matrix using the Direct Linear Transformation (DLT) algorithm. This requires a set of known 2D-3D point correspondences, which are derived by combining pixel locations from the RGB image with depth values from the depth map, using the camera's known intrinsic parameters. The implementation is encapsulated within a Python 'Camera' class for modularity and clarity.

2 Methodology

The process involves establishing point correspondences and solving a system of linear equations to find the matrix that best describes the projection.

2.1 From Pixels to World Coordinates

Given an RGB image and a depth map, we can calculate the real-world 3D coordinates (X, Y, Z) for any pixel (u, v) if the camera's intrinsic parameters are known. The relationship is defined by the inverse pinhole camera model:

$$X = \frac{(v - c_x) \cdot Z_d}{f_x} \tag{1}$$

$$Y = \frac{(u - c_y) \cdot Z_d}{f_y} \tag{2}$$

$$Z = Z_d \tag{3}$$

where (u, v) are the pixel coordinates (row, column), (c_x, c_y) is the principal point, (f_x, f_y) are the focal lengths, and Z_d is the depth value at that pixel. This step allows us to build a set of 2D-3D point correspondences.

2.2 The Direct Linear Transformation (DLT) Algorithm

The DLT algorithm provides a linear method for finding the projection matrix M. A 3D point $P_w = [X, Y, Z, 1]^T$ is projected to a 2D image point $p_i = [u, v, 1]^T$ by the equation $p_i \cong MP_w$. By expanding this relationship, we derive two linear equations for each point correspondence.

For n correspondences, we construct a $2n \times 12$ matrix A such that $Ap = 0$, where p is a 12-element vector containing the flattened entries of M.

$$A = \begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix}$$

The optimal solution for p is found by performing Singular Value Decomposition (SVD) on matrix A. The solution corresponds to the singular vector associated with the smallest singular value, which is the last row of the V^T matrix.

2.3 Validation via Reprojection Error

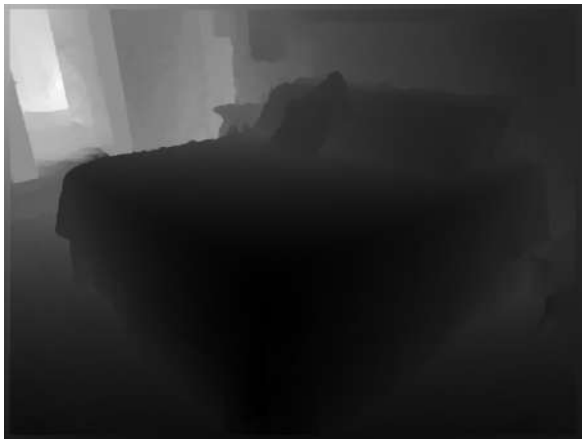
To validate the accuracy of our estimated matrix M, we reproject the original 3D world points back onto the 2D image plane. The mean reprojection error is the average Euclidean distance between the original pixel coordinates (u, v) and the reprojected coordinates (u', v'). A small error indicates a successful calibration.

3 Experimental Setup & Results

3.1 Input Data and Camera Parameters

For our analysis, we used an indoor scene from the NYU Depth V2 dataset. The source RGB image and its corresponding depth map are shown in Figure 1. From this data, a total of 306,661 pixels with valid depth information were found.

(a) Source RGB Image



(b) Corresponding Depth Map

Figure 1: The input data used for camera calibration.

The provided intrinsic camera parameters for this dataset are listed in Table 1.

Table 1: Intrinsic Camera Parameters

Parameter	**Value**
f_x (Focal Length, x-axis)	518.8579
f_y (Focal Length, y-axis)	519.4696
c_x (Principal Point, x-axis)	325.5824
c_y (Principal Point, y-axis)	253.7362

3.2 Estimated Projection Matrix (M)

Using $n = 10$ randomly sampled correspondence points, the DLT algorithm yielded the following estimated 3x4 projection matrix, presented here in scientific notation:

$$M_{est} = \begin{bmatrix} -6.160 \times 10^{-1} & -4.314 \times 10^{-15} & -3.865 \times 10^{-1} & -1.222 \times 10^{-13} \\ 2.189 \times 10^{-15} & -6.167 \times 10^{-1} & -3.012 \times 10^{-1} & -3.312 \times 10^{-13} \\ -9.801 \times 10^{-17} & 4.662 \times 10^{-17} & -1.187 \times 10^{-3} & -7.212 \times 10^{-16} \end{bmatrix}$$

3.3 Validation and Visualization

The quality of the estimated matrix M was evaluated by calculating the mean reprojection error for the 10 sample points. The calculated error was 1.5484×10^{-11} pixels. This infinitesimally small error confirms the high fidelity of the estimated matrix and the correctness of our implementation.

Figure 2 provides a visual confirmation of this result. The original sampled points (red circles) are overlaid with their reprojected counterparts (lime crosses), showing a perfect match and validating the accuracy of the computed matrix.

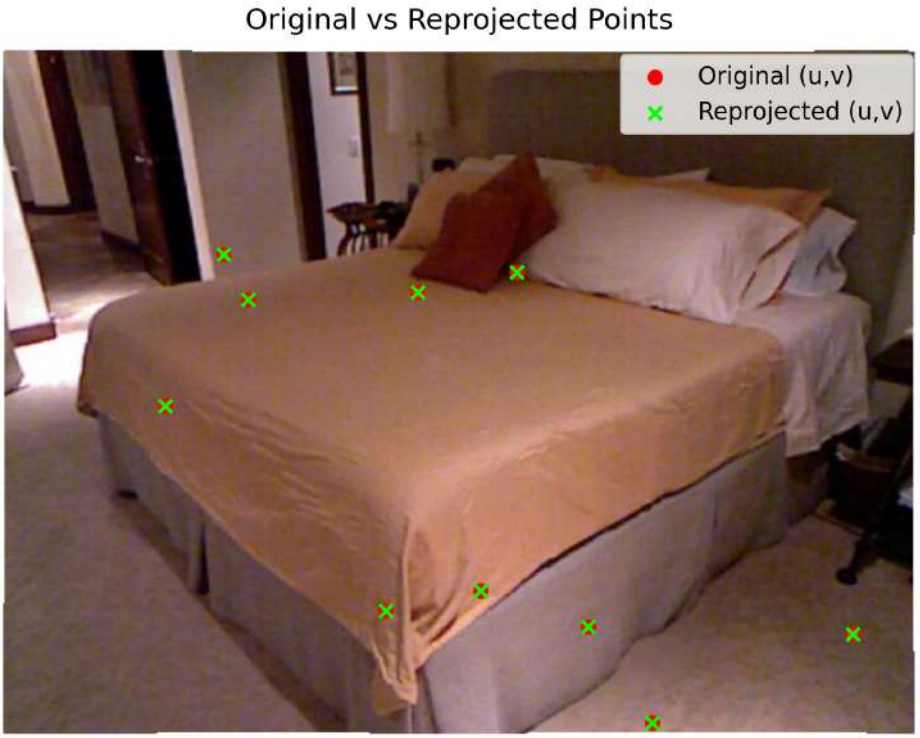


Figure 2: Visualization of original sampled points vs. reprojected points using the estimated matrix M. The complete overlap demonstrates the high accuracy of the calibration.

4 Conclusion

The implementation of a class-based camera model successfully produced a valid 3x4 perspective projection matrix. The extremely low mean reprojection error, visually confirmed by the overlapping points in the final plot, demonstrates that the DLT algorithm is a powerful and effective method for camera calibration when accurate 2D-3D correspondences are available. This project reinforces the core concepts of geometric transformations that bridge the gap between our 3D world and 2D images.

A Full Python Code

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

# --- 1. Data Preparation ---
rgb_image = cv2.imread('img.png')
depth_image_bgr = cv2.imread('dep.png')
depth_data = depth_image_bgr[:, :, 0].astype(np.float32)

# --- 2. Camera Calibration Implementation ---
class Camera:
    """A class to represent a camera and handle its calibration."""
    def __init__(self, fx, fy, cx, cy):
        """Initializes the camera with its intrinsic parameters."""
        self.fx = fx
        self.fy = fy
        self.cx = cx
        self.cy = cy
        self.projection_matrix = None

    def project_to_3d(self, u, v, depth_value):
        """Projects a single 2D pixel (u, v) to a 3D point (X, Y, Z)."""
        Z = depth_value
        X = (v - self.cx) * Z / self.fx
        Y = (u - self.cy) * Z / self.fy
        return np.array([X, Y, Z])

    def calibrate(self, image_points, world_points):
        """Computes the projection matrix using the DLT algorithm."""
        if len(image_points) < 6:
            raise ValueError("At least 6 point correspondences are required.")

        A = []
        for i in range(len(image_points)):
            u, v = image_points[i]
            X, Y, Z = world_points[i]

            row1 = [X, Y, Z, 1, 0, 0, 0, 0, -v*X, -v*Y, -v*Z, -v]
            row2 = [0, 0, 0, 0, X, Y, Z, 1, -u*X, -u*Y, -u*Z, -u]

            A.append(row1)
            A.append(row2)

        A = np.array(A)
        _, _, Vt = np.linalg.svd(A)
        p = Vt[-1]
        self.projection_matrix = p.reshape(3, 4)
        return self.projection_matrix

    def calculate_reprojection_error(self, image_points, world_points):
        """Calculates the average reprojection error."""
        if self.projection_matrix is None:
            raise RuntimeError("Camera is not calibrated yet.")

        total_error = 0
        num_pts = len(image_points)

        for i in range(num_pts):
            P_world = np.append(world_points[i], 1)
            p_image_homogeneous = self.projection_matrix @ P_world
            p_image_reprojected = p_image_homogeneous / p_image_homogeneous[2]
```

```
62
63              u_reproj, v_reproj = p_image_reprojected[1], p_image_reprojected[0]
64              u_orig, v_orig = image_points[i]
65
66              error = np.sqrt((u_reproj - u_orig)**2 + (v_reproj - v_orig)**2)
67              total_error += error
68
69          return total_error / num_pts
70
71  # --- 3. Model Execution and Analysis ---
72  nyu_camera = Camera(
73      fx=5.1885790117450188e+02,
74      fy=5.1946961112127485e+02,
75      cx=3.2558244941119034e+02,
76      cy=2.5373616633400465e+02
77  )
78
79  valid_coords = np.argwhere(depth_data > 0)
80  num_valid_points = len(valid_coords)
81  print(f"Found {num_valid_points} pixels with valid depth information.")
82
83  num_samples = 10
84  random_indices = np.random.choice(num_valid_points, num_samples, replace=False)
85  sampled_pixel_coords = [tuple(valid_coords[i]) for i in random_indices]
86  sampled_world_coords = [nyu_camera.project_to_3d(u, v, depth_data[u, v]) for u,
        v in sampled_pixel_coords]
87
88  estimated_M = nyu_camera.calibrate(sampled_pixel_coords, sampled_world_coords)
89  error = nyu_camera.calculate_reprojection_error(sampled_pixel_coords,
        sampled_world_coords)
90
91  print(f"\\nMean Reprojection Error: {error:.4e} pixels\\n")
92  print("Estimated Projection Matrix M:\\n", estimated_M)
93
94  # --- Visualization ---
95  image_points_arr = np.array(sampled_pixel_coords)
96  world_points_arr = np.array(sampled_world_coords)
97  world_points_homogeneous = np.hstack((world_points_arr, np.ones((num_samples,
        1))))
98
99  reprojected_homogeneous = estimated_M @ world_points_homogeneous.T
100 reprojected_coords = reprojected_homogeneous / reprojected_homogeneous[2, :]
101
102 plt.figure(figsize=(10, 8))
103 plt.imshow(cv2.cvtColor(rgb_image, cv2.COLOR_BGR2RGB))
104 plt.scatter(image_points_arr[:, 1], image_points_arr[:, 0], c='red', marker='o'
        , s=80, label='Original Points')
105 plt.scatter(reprojected_coords[0, :], reprojected_coords[1, :], c='lime',
        marker='x', s=80, label='Reprojected Points')
106 plt.title('Visual Confirmation of Calibration Accuracy')
107 plt.legend()
108 plt.axis('off')
109 plt.show()
```

Listing 1: Full Python script for camera calibration.